# ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/19/26

Aisha Shen

## Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

## Set up

```
#Load/install required package here
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.5.2
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
library(Kendall)
```

```
## Warning: package 'Kendall' was built under R version 4.5.2
```

```r
library(tseries)
```

```
## Warning: package 'tseries' was built under R version 4.5.2
```

```r
library(outliers)
library(tidyverse)
```

```
## Warning: package 'tibble' was built under R version 4.5.2

## Warning: package 'tidyr' was built under R version 4.5.2

## Warning: package 'readr' was built under R version 4.5.2

## Warning: package 'purrr' was built under R version 4.5.2

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr   1.1.4     v stringr 1.6.0
## v forcats 1.0.1     v tibble  3.3.1
## v purrr   1.2.1     v tidyr   1.3.2
## v readr   2.1.6

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
#install.packages("smooth")
library(smooth)
```

```
## Warning: package 'smooth' was built under R version 4.5.2

## Loading required package: greybox

## Warning: package 'greybox' was built under R version 4.5.2

## Package "greybox", v2.0.8 loaded.
##
##
## Attaching package: 'greybox'
##
## The following object is masked from 'package:tidyr':
```

```
##
##     spread
##
## The following object is masked from 'package:lubridate':
##
##     hm
##
## This is package "smooth", v4.4.0
```

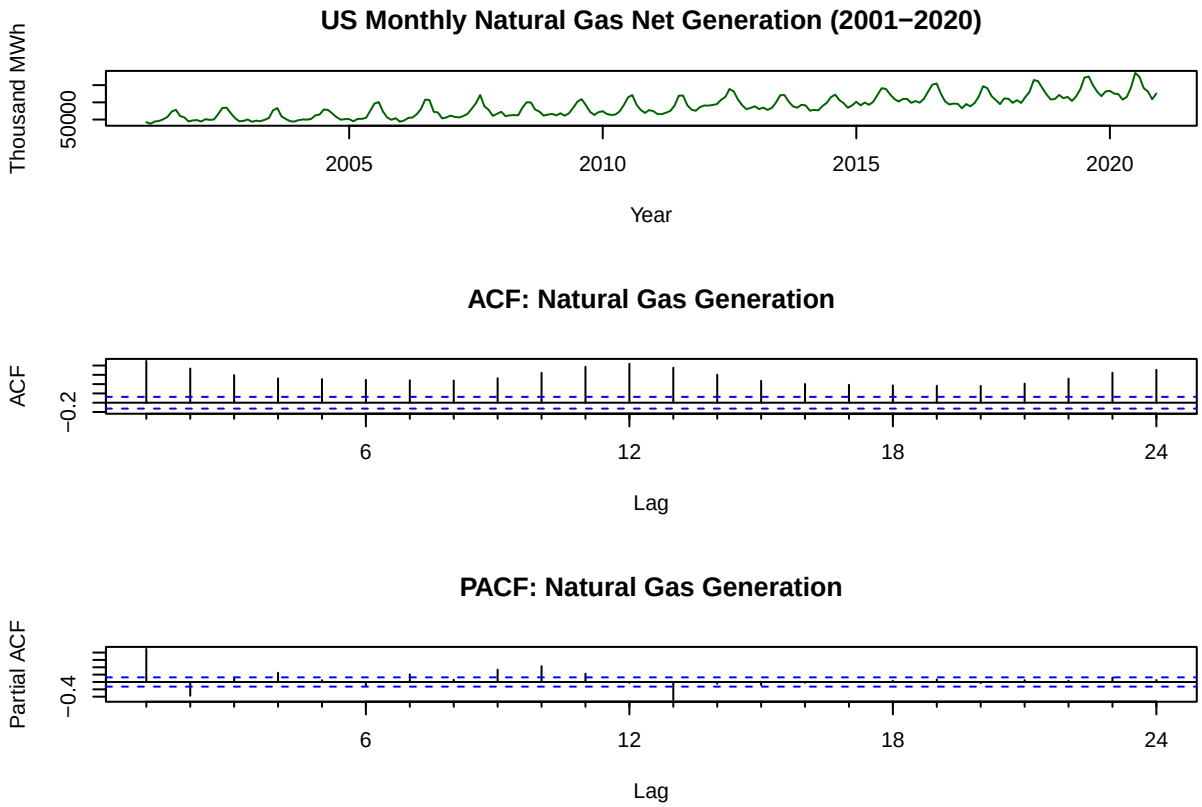
## Importing and processing the data set

Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

### Q1

Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

```r
electricity_generation <- read.csv(file="~/Documents/TimeSeries/TSA_Sp26/Data/Net_generation_United_Sta

electricity_generation <- electricity_generation[nrow(electricity_generation):1, ]

ng_column_name <- "natural.gas.thousand.megawatthours"

ng_ts <- ts(electricity_generation[[ng_column_name]],
            start = c(2001, 1),
            frequency = 12)

par(mfrow = c(3, 1))

plot(ng_ts,
     main = "US Monthly Natural Gas Net Generation (2001-2020)",
     ylab = "Thousand MWh",
     xlab = "Year",
     col = "darkgreen")

# ACF Plot (Autocorrelation Function)
Acf(ng_ts, main = "ACF: Natural Gas Generation")

# PACF Plot (Partial Autocorrelation Function)
Pacf(ng_ts, main = "PACF: Natural Gas Generation")
```



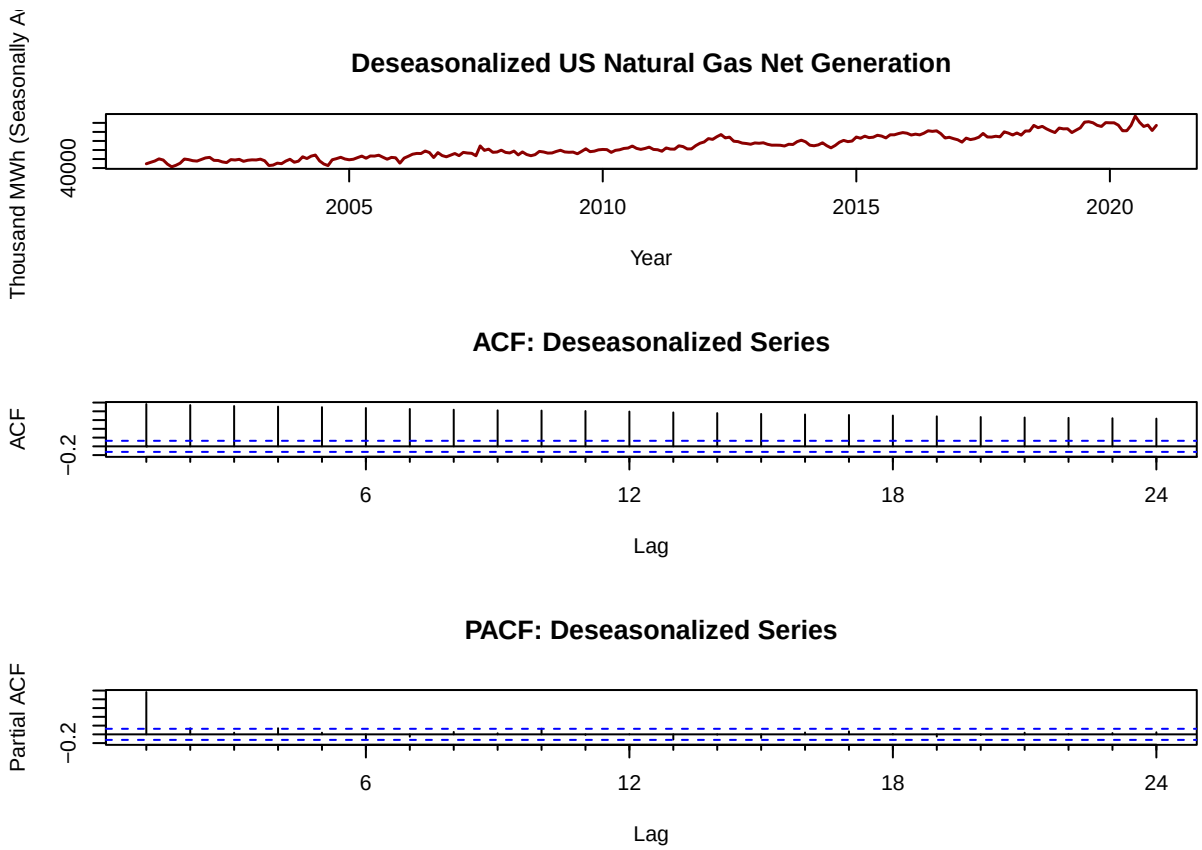

```r
# Reset plotting layout
par(mfrow = c(1, 1))
```

### Q2

Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

```r
ng_decomp <- decompose(ng_ts, type = "additive")
ng_deseason <- seasadj(ng_decomp)
par(mfrow = c(3, 1))

# Time Series Plot (Deseasonalized)
plot(ng_deseason,
     main = "Deseasonalized US Natural Gas Net Generation",
     ylab = "Thousand MWh (Seasonally Adj.)",
     xlab = "Year",
     col = "darkred",
     lwd = 1.5)

# ACF Plot
Acf(ng_deseason, main = "ACF: Deseasonalized Series")
```

```r
# PACF Plot
Pacf(ng_deseason, main = "PACF: Deseasonalized Series")
```




```r
# Reset layout
par(mfrow = c(1, 1))
```

Time Series Plot: In Q1, the plot shows distinct peaks and troughs occurring every year. The deseasonalized plot is much smoother; the yearly cycles are removed, making the long-term upward trend of natural gas usage much more apparent. ACF Plot: The ACF from Q1 shows significant spikes at lags 12, 24, and 36 (indicating seasonality). In the deseasonalized ACF, these seasonal spikes are removed and we see high autocorrelation at short lags (1, 2, 3...) because the trend and cycle components still exist.PACF Plot: The PACF in Q1 shows a sharp spike at lag 1 and some small spikes at other intervals In the deseasonalized version, we only see the correlation structure related to the trend at a lag of 1.

## Modeling the seasonally adjusted (i.e., the deseasonalized series)

### Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

```r
adf_result <- adf.test(ng_deseason, alternative = "stationary")
```

```
## Warning in adf.test(ng_deseason, alternative = "stationary"): p-value smaller
## than printed p-value
```

```r
print(adf_result)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ng_deseason
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

The ADF test is for stationarity. The null hypothesis is that the series is non-stationary while the alternate hypothesis is that the series is stationary. The test result (p<0.05) suggests the deseasonalized series is indeed stationary as we reject the null hypothesis.

```r
mk_result <- MannKendall(ng_deseason)
print(mk_result)
```

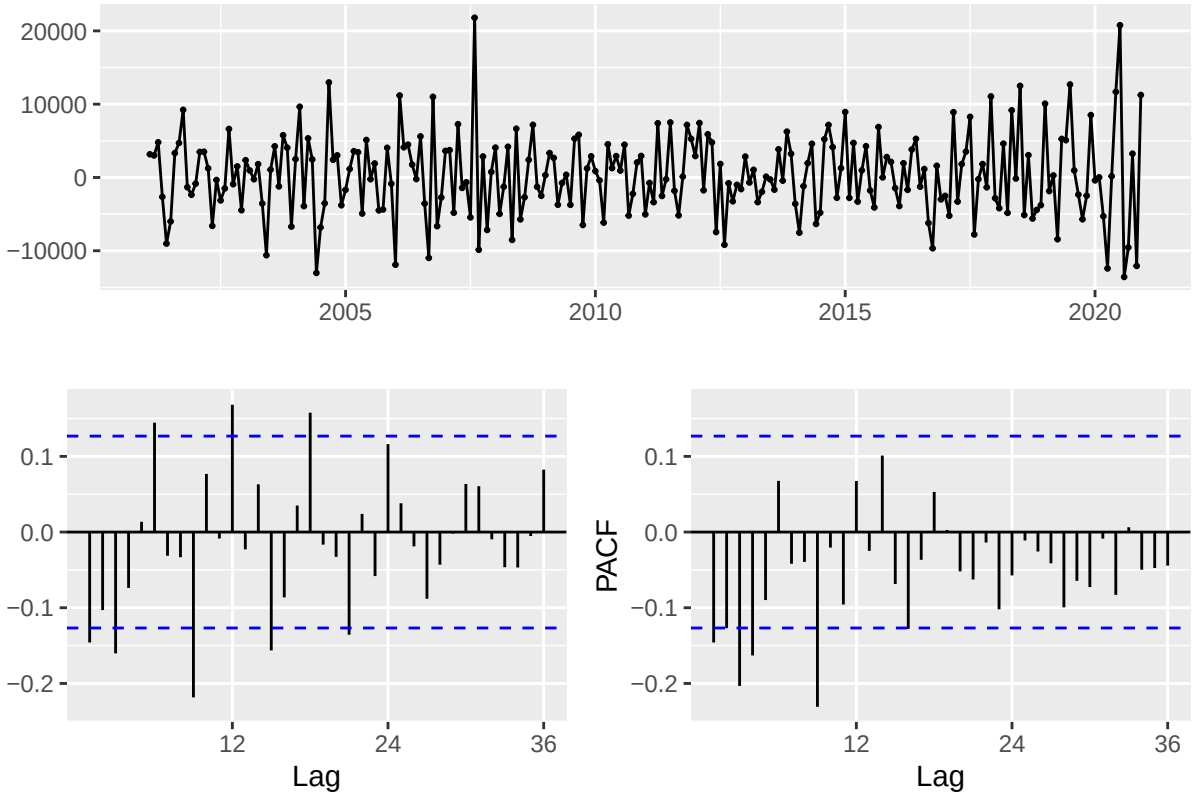
```
## tau = 0.843, 2-sided pvalue =< 2.22e-16
```

The Mann Kendall Test checks for monotonic trend in a time series.The null hypothesis is that the series has no clear trend while the alternate hypothesis is that the series has a monotonic trend. The test confirms that there is a statistically significant upward trend, proving that the increase in natural gas usage is not just random noise .

### Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters $p, d$ and $q$. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

```r
diff_ng <- diff(ng_deseason)
ggtsdisplay(diff_ng)
```




From the MannKendall test we know that there is a clear upward trend and that the mean is changing. To remove the strong linear trend identified by the Mann-Kendall test, we must take the first difference of the data. We use the ndiffs function for this to get the differenced time series. Looking at the plot of the first differenced series it appears to be white noise so we can draw inference that d=1. Based only on the plots of Q2 we can see that the PACF shows a sharp spike at lag 1,while the ACF decays slowly. Based on this, we can infer that that q could be 1 and p could also be 1.

In practice a better approach is to consider a range of models and observe the one that gives the lowest AIC. We can shortlist these based on the observed plot of the ACF and PACF of the differenced time series which shows spikes at 1, 2 but also significantly greater at 3. However, since we would like to analytically select a model based on just prior plots before having seen the differenced time series we select the best model to be ARIMA(1,1,1)

```
fit111 <- Arima(ng_deseason, order=c(1,1,1), include.drift = TRUE)
fit011 <- Arima(ng_deseason, order=c(0,1,1), include.drift = TRUE)
fit210 <- Arima(ng_deseason, order=c(2,1,0), include.drift = TRUE)
fit313 <- Arima(ng_deseason, order=c(3,1,3), include.drift = TRUE)

# Compare their AIC values
fit111$aic
```

```
## [1] 4774.213
```

```r
fit011$aic
```

```
## [1] 4796.664
```

```r
fit210$aic
```

```
## [1] 4797.215
```

```r
fit313$aic
```

```
## [1] 4762.144
```

> NOTE: Based on these results we could have selected a ARIMA(3,1,3) model as the best model however we will use ARIMA(1,1,1) for subsequent parts since that is the best choice based on Q2 and Q3.

### Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

```r
print(fit111 )
```

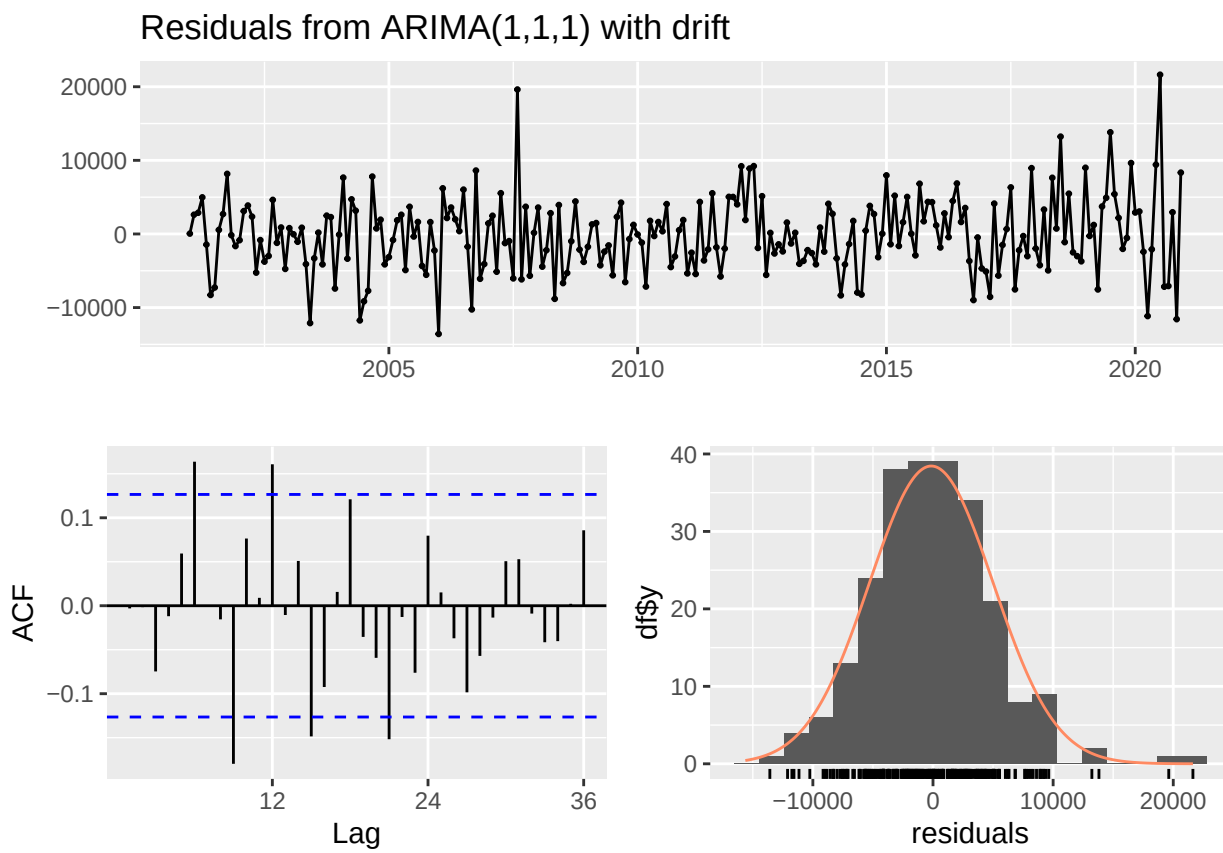
```
## Series: ng_deseason
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

### Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

## Modeling the original series (with seasonality)

```r
checkresiduals(fit111)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1) with drift
## Q* = 48.356, df = 22, p-value = 0.0009736
##
## Model df: 2.   Total lags used: 24
```

The residuals do appear to be white noise based on the time series plot of residuals but there are some bursts of volatility. This is also seen in the histogram where a right tail is observed, though it is mostly a Gaussian centred on 0 (which is what we would expect if this is the correctly specified model)

### Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., $P$, $D$ and $Q$.

```r
adf_result <- adf.test(ng_ts, alternative = "stationary")
```

```
## Warning in adf.test(ng_ts, alternative = "stationary"): p-value smaller than
## printed p-value
```

```r
print(adf_result)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ng_ts
## Dickey-Fuller = -8.9602, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
mk_result <- MannKendall(ng_ts)
print(mk_result)
```

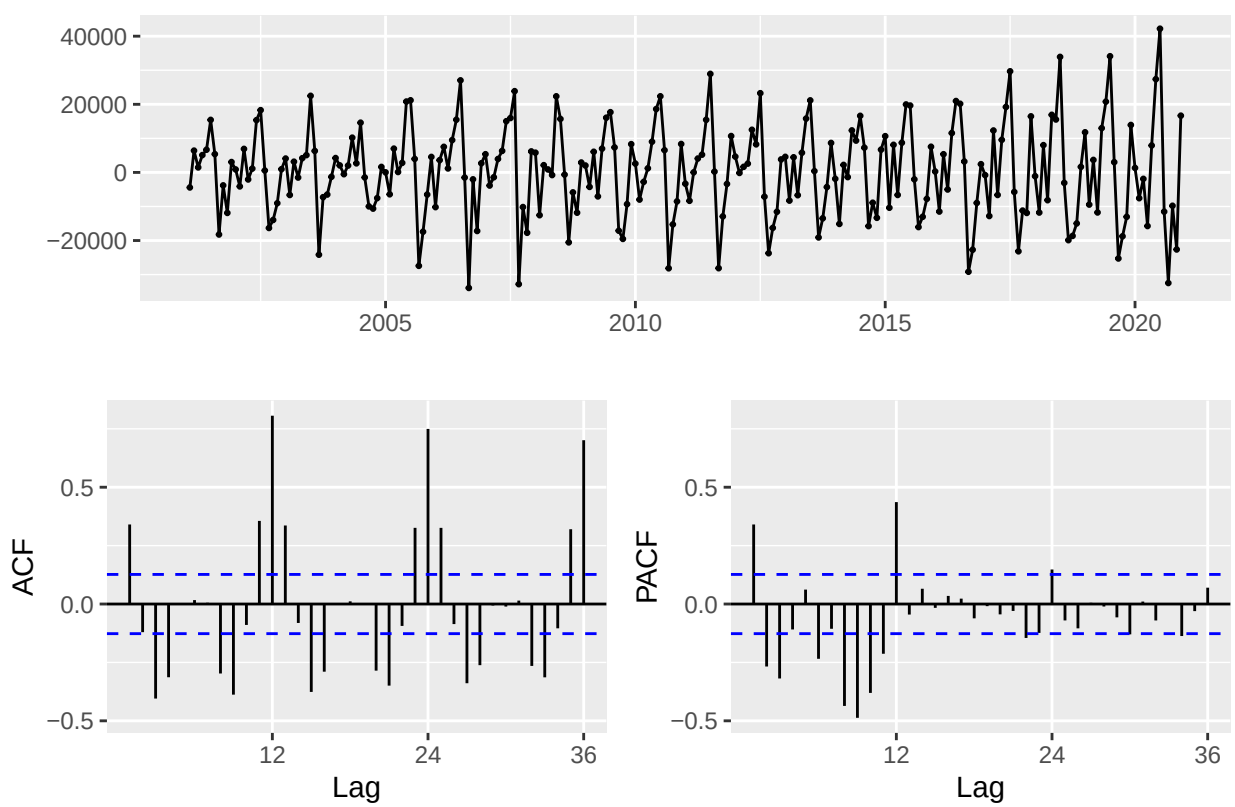
```
## tau = 0.651, 2-sided pvalue =< 2.22e-16
```

From the ADF test we can reject the null and so the series is stationary, and from Mann Kendall we can reject the null and so the series has a strong monotonic upward trend. Now we can difference as before

```r
ns_diff <- nsdiffs(ng_ts)
cat("Number of seasonal differencing needed: ",ns_diff)
```

```
## Number of seasonal differencing needed:  1
```

```r
diff_ng <- diff(ng_ts)
ggtsdisplay(diff_ng)
```



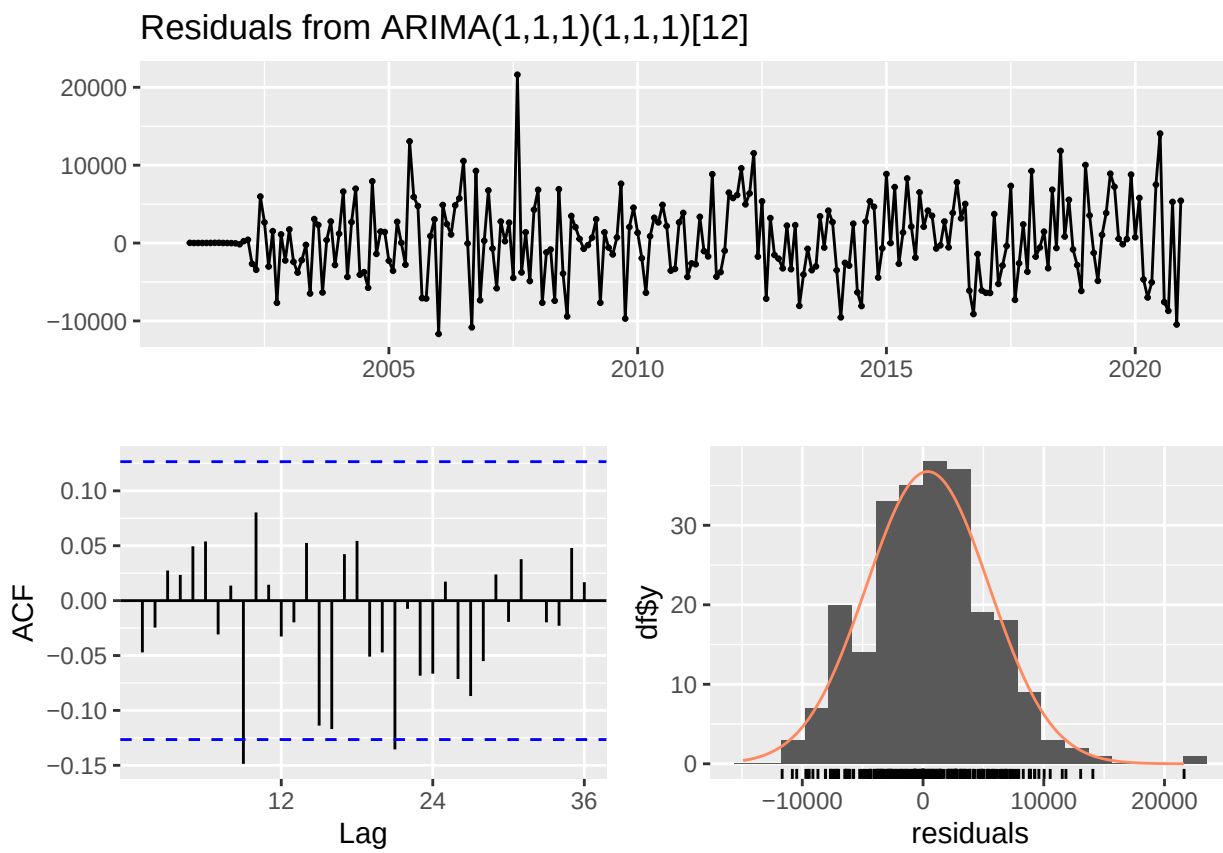
> From this differenced plot, we can clearly see an annual trend showing the 12 month periodicity. The ACF and PACF also spike at 1 respectively so we can infer p and q to be 1.

```r
fit_seasonal <- Arima(ng_ts,
                      order = c(1, 1, 1),
                      seasonal = list(order = c(1, 1, 1), period = 12))
```

```r
summary(fit_seasonal)
```

```
## Series: ng_ts
## ARIMA(1,1,1)(1,1,1)[12]
##
## Coefficients:
##          ar1      ma1     sar1     sma1
##       0.7328  -0.9819  -0.0202  -0.6909
## s.e.  0.0504   0.0182   0.0949   0.0772
##
## sigma^2 = 28044713:  log likelihood = -2272.17
## AIC=4554.35   AICc=4554.62   BIC=4571.47
##
## Training set error measures:
##                     ME     RMSE      MAE        MPE     MAPE      MASE
## Training set 389.1661 5104.725 3965.388 0.02834762 4.662867 0.4848005
##                      ACF1
## Training set -0.04714322
```

```r
checkresiduals(fit_seasonal)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(1,1,1)[12]
## Q* = 27.654, df = 20, p-value = 0.1179
##
## Model df: 4.   Total lags used: 24
```

Looking at the plot of the residuals we can see that we have explained away the periodicity in the ACF spikes at intervals of ### Q8

Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.

If you compare these directly, ARIMA looks better than the seasonal ARIMA. But **you cannot directly compare** ARIMA on deseasonalized series with SARIMA on original series because they are modeling different dependent variables. ARIMA residuals look reasonably white, while SARIMA appears to be heteroskedastic with periods of high and low variance. Since our goal is to forecast the original time series SARIMA is more coherent. Because: * It models trend and seasonality jointly * It keeps the stochastic seasonal structure * No need to manually re-add seasonal component Ultimately, SARIMA uses the raw data while ARIMA uses a processed version of the data so the gain in performance is only an artefact of the preprocessing as opposed to better model selection.

## Checking your model with the auto.arima()

**Please** do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

### Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

```r
auto.arima(ng_deseason)
```

```
## Series: ng_deseason
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

> The order of the best ARIMA model is ARIMA(1,1,1) which is the same as what we identified in Q4.

### Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?

```r
auto.arima(ng_ts)
```

```
## Series: ng_ts
## ARIMA(1,0,0)(0,1,1)[12] with drift
##
## Coefficients:
##          ar1     sma1     drift
##       0.7416  -0.7026  358.7988
## s.e.  0.0442   0.0557   37.5875
##
## sigma^2 = 27569124:  log likelihood = -2279.54
## AIC=4567.08   AICc=4567.26   BIC=4580.8
```

> We get a AR(1) model with seasonal moving average and a seasonality of 12 months along with a drift component.